

Overtune 3

Step-by-Step DSP Tutorials & Implementation Guides

Practical Synthesis Workflows, Audio Presets, and Firmware Deployment

Overtune Audio DSP Engineering Team

September 30, 2026

Abstract

This guide provides practical step-by-step engineering tutorials for designing, verifying, and deploying digital filters using **Overtune 3**. Topics include audio band splitting, notch filtering for electrical interference, anti-aliasing reconstruction, pole-zero stability auditing, and multi-platform code export for real-time DSP systems.

1 Filter Design Workflow Overview

Designing a high-performance digital filter in Overtune 3 follows a structured four-stage methodology:

1. **Specification:** Define passband ripple δ_{pass} , stopband attenuation δ_{stop} , sampling rate F_s , and critical cutoff frequencies (F_{c1}, F_{c2}).
2. **Synthesis:** Select an analog prototype approximation (Butterworth, Chebyshev I/II, Elliptic, Bessel) and calculate pre-warped bilinear transform coefficients.
3. **Diagnostic Verification:** Audit frequency magnitude, phase unwrapping, group delay transit time, and pole-zero stability (—poles— < 1.0).
4. **Deployment:** Export optimized C, C++20, or Python source code directly into production audio applications and embedded microcontrollers.

2 Standard Audio Engineering Tutorials

2.1 Tutorial 1: Audio Sub-Bass Cut (30 Hz Highpass)

Objective: Remove DC offset, mic stand thumps, and turntable rumble below the audible music threshold while avoiding transient smearing in kick drums.

- **Topology:** High-Pass Filter (HPF)
- **Response:** Butterworth (Order 4, -24 dB/octave)
- **Parameters:** $F_s = 48,000$ Hz, $F_c = 30$ Hz
- **Verification:** Confirm magnitude is -3.0 dB at 30 Hz, with monotonic roll-off reaching -40 dB below 10 Hz.

2.2 Tutorial 2: Mains Hum Interference Notch (50/60 Hz Bandstop)

Objective: Reject AC power line induction noise with razor-sharp rejection (> 50 dB) without attenuating adjacent musical bass frequencies.

- **Topology:** Band-Stop / Notch Filter (BSF)

- **Response:** Elliptic or Chebyshev II (Order 4 or 6)
- **Parameters:** $F_s = 48,000$ Hz, $F_{c1} = 48$ Hz, $F_{c2} = 52$ Hz, $\delta_{\text{stop}} = 50$ dB
- **Mechanism:** Places complex transmission zeros directly on the unit circle $e^{\pm j2\pi(50)/48000}$, generating infinite mathematical nulls at the line frequency.

2.3 Tutorial 3: Speech / Telephony Bandpass (300 Hz – 3.4 kHz)

Objective: Synthesize an ITU-T G.712 compliant voice channel filter that isolates human speech fundamentals and formants while attenuating low-frequency room noise and high-frequency digital hash.

- **Topology:** Band-Pass Filter (BPF)
- **Response:** Chebyshev Type I (Order 4, 0.5 dB passband ripple)
- **Parameters:** $F_s = 8,000$ Hz or $48,000$ Hz, $F_{c1} = 300$ Hz, $F_{c2} = 3,400$ Hz
- **Q-Factor:** Quality factor $Q = f_0/\text{BW} = \sqrt{300 \cdot 3400}/(3400 - 300) \approx 0.32$.

2.4 Tutorial 4: Anti-Aliasing Reconstruction Filter (20 kHz Lowpass)

Objective: Steep brickwall cutoff for Hi-Res audio DAC/ADC conversions protecting the Nyquist boundary (24 kHz at 48 kHz sampling).

- **Topology:** Low-Pass Filter (LPF)
- **Response:** Elliptic (Order 8, 0.1 dB passband ripple, 70 dB stopband rejection)
- **Parameters:** $F_s = 48,000$ Hz, $F_c = 20,000$ Hz
- **Transition Band:** Drops from -0.1 dB at 20 kHz to -70 dB at 22 kHz.

3 Diagnostic Analysis & Stability Auditing

Before deploying any filter, inspect the following plots in the **Analysis Suite** (Ctrl+2):

- **Pole-Zero Constellation:** Ensure all poles satisfy $\max_k |p_k| < 1.0$. Highly resonant filters (high Q) have poles positioned close to the unit circle (e.g. $|p| \approx 0.985$); verify sufficient numerical precision.
- **Group Delay τ_g :** High-order Chebyshev and Elliptic filters show peaking group delay near the cutoff frequency, introducing dispersion. For transient-sensitive audio, use Bessel or lower orders.
- **Impulse & Step Responses:** Check for excessive ringing, overshoot, or slow settling time.

4 Exporting and Embedding into Production Firmware

Navigate to the **Production Code Exporter** (Ctrl+4) to generate deployment source:

- **Embedded C99:** Generates static coefficient arrays `double b[N][3], a[N][3]` and an in-lined Direct Form II Transposed processing loop with zero dynamic memory allocation.
- **Modern C++20:** Generates a templated `BiquadCascade<N>` class featuring `constexpr` coefficient storage, SIMD auto-vectorization friendliness, and thread-safe sample processing.
- **Python SciPy:** Generates an executable script utilizing `scipy.signal.sosfilt` for quick offline batch processing and research verification.

5 References & Tutorial Resources

- [Audio Filter Design Guide — Wikipedia](#)
- [Direct Form II Transposed Implementation — Wikipedia](#)
- [Low-Pass Filter Applications — Wikipedia](#)
- [High-Pass Filter Applications — Wikipedia](#)
- [Band-Pass Filter Applications — Wikipedia](#)
- [Overtune 3 GitHub Repository & DSP Source Code](#)